

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

УДК 004.75

Отчет об исследовательском проекте на тему:
Системы балансировки нагрузки в гибридных распределенных системах
(промежуточный, этап 1)

Выполнил студент:

группы #БПМИ236, 3 курса

Разухин Александр Сергеевич

Принял руководитель проекта:

Курочкин Илья Ильич

к.т.н., зав.лаб. Ц-1

ИППИ РАН

Москва 2026

Содержание

Аннотация	3
1 Введение	4
1.1 Описание предметной области	4
1.2 Релевантность и значимость	4
1.3 Цели и задачи	5
1.4 Метрики оценки производительности	5
1.5 Предполагаемые результаты проекта	6
2 Литературный обзор	6
2.1 Обзор предметной области	6
3 Обзор существующих методов	8
3.1 Сравнение подходов	9
3.2 Выводы из обзора	9
Список литературы	10

Аннотация

При проектировании высокопроизводительных и распределенных систем одной из важнейших задач является выбор и настройка системы выдачи заданий (балансировщик нагрузки). При использовании гомогенных вычислительных узлов в кластере можно использовать готовые решения (slurm, torque и др.), но в гетерогенных системах готовые решения не будут работать из коробки. В распределенных системах из-за большой гетерогенности и других ограничений может возникнуть потребность разработки оригинального планировщика заданий. В распределенной вычислительной системе в качестве части ресурсов могут быть использованы облачные ресурсы или ресурсы кластеров. Уже были разработаны решения (к примеру, CluBoRun в грид-системе на платформе BOINC) по использованию ресурсов кластера в фоновом режиме. Предлагается разработать /модифицировать систему выдачи заданий для грид-системы на платформе BOINC, в которой присутствуют сегменты кластеров.

Ключевые слова

Распределенные системы, балансировка нагрузки

1 Введение

1.1 Описание предметной области

Предметная область проекта охватывает распределенные системы с гетерогенной инфраструктурой с основными компонентами:

- BOINC (Berkeley Open Infrastructure for Network Computing) — платформа для добровольных вычислений, объединяющая ресурсы разнородных устройств (серверов, кластеров, персональных компьютеров) в единую сеть
- Кластеры — узлы, интегрированные в распределенную систему с целью повышения доступности
- Гетерогенные ресурсы — устройства с различными характеристиками: вычислительной мощностью, объемом памяти, доступностью и другими
- Балансировщик нагрузки — компонент, отвечающий за распределение вычислительных задач между узлами с учетом их возможностей и текущих состояний

Также предметная область имеет следующие особенности:

- Отсутствие централизованного контроля за узлами системы
- Разнородность ресурсов каждого из узлов
- Различные режимы работы узлов (не все устройства доступны на протяжении 100% времени)
- Различный объем задач, время выполнения которых может варьироваться от нескольких минут до нескольких дней
- Необходимость минимизации накладных расходов для балансировщика

1.2 Релевантность и значимость

Актуальность данного проекта обусловлена множеством существующих на данный момент проблем, таких как:

- 1 Современные вычислительные задачи такие, как анализ больших данных, биоинформатика, требуют огромного количества ресурсов, что способствует росту числа гетерогенных распределенных систем

- 2 Стандартные планировщики задач оптимизированы под гомогенные системы и не учитывают различия в производительности узлов и их доступность, что в гетерогенных системах приводит к простаиванию некоторых узлов или, наоборот, излишней загруженности отдельных
- 3 Необходимость развития динамических алгоритмов балансировки в противовес существующим решениям с их статическими правилами
- 4 Оптимизация всех вычислений, выполняющихся на платформе BOINC

1.3 Цели и задачи

Целью данного проекта является разработка системы выдачи заданий для распределенной системы на платформе BOINC, в которой присутствуют сегменты кластеров. Для ее достижения необходимо было выполнить следующие задачи:

- 1 Обзор систем балансировки нагрузки для кластеров и распределенных систем
- 2 Выбор системы моделирования и адаптация модели под специфику гибридной распределенной системы
- 3 Реализация/модификация системы балансировки нагрузки гибридной распределенной системы
- 4 Тестирование системы балансировки на модели гибридной распределенной системы

1.4 Метрики оценки производительности

Выбор метрик обусловлен необходимостью учета как общих показателей производительности, так и особенностей гетерогенной вычислительной среды.

- Пропускная способность (Throughput) – число выполненных задач за единицу времени. Формула: $T = \frac{N_{\text{completed}}}{\Delta t}$.
- Время выполнения задачи – среднее, медиана, перцентили (95%, 99%). Для каждой задачи i : $R_i = t_{\text{finish}}^{(i)} - t_{\text{arrival}}^{(i)}$. Среднее: $\bar{R} = \frac{1}{N} \sum_i R_i$.
- Загруженность узлов – для узла j : $u_j = \frac{\text{busy_cpu_time}_j}{\text{observation_time}}$. Сводные метрики: среднее по узлам $\bar{u}$ и коэффициент вариации $CoV = \frac{\sigma(u_j)}{\bar{u}}$ (показывает балансировку).

- Общее время завершения набора задач – $C = \max_i \{t_{\text{finish}}\} - \min_i \{t_{\text{arrival}}\}$
- Накладные расходы планировщика – среднее время, затраченное на принятие решения по задаче, и доля от общего времени:

$$OH_{\text{avg}} = \frac{1}{N} \sum_i t_{\text{sched}}^{(i)}, \quad OH\% = \frac{\sum_i t_{\text{sched}}^{(i)}}{T_{\text{wall}}} \cdot 100\%.$$

Такой набор метрик позволяет оценить как производительность системы, так и качество балансировки нагрузки в условиях гетерогенной среды.

1.5 Предполагаемые результаты проекта

- 1 Подробное описание методики моделирования гибридной распределенной системы
- 2 Подробное описание системы балансировки нагрузки гибридной распределенной системы
- 3 Анализ результатов моделирования системы
- 4 Доклад о проекте на научной конференции
- 5 Написание и публикация по итогам проделанной работы

2 Литературный обзор

2.1 Обзор предметной области

В своем начале Интернет имел очень мало потребителей, так что любой сервер мог выдерживать нагрузку даже самого популярного сайта, к тому же его падение не стало бы большой проблемой. Со временем люди были вынуждены заняться вопросами масштабирования ввиду растущей нагрузки. И если сначала проблема решалась увеличением числа процессоров, их улучшением, увеличением объема оперативной памяти, то после стало понятно, что есть потолок такого вертикального масштабирования, так что появилось понятие балансировки нагрузки. [2]

В общем случае, порядок работы балансировщика такой:

- 1 Получение входящих запросов от различных клиентов
- 2 Расчет размера входящих запросов и формирование очереди запросов
- 3 Периодическая проверка состояния серверов в пуле с помощью системы мониторинга

- 4 Использование стратегии / алгоритма / эвристики для выбора подходящего сервера под каждый входящий запрос

Сама идея балансировки нагрузки вносит в систему большое количество преимуществ:

- 1 Упрощенное отслеживания трафика
- 2 Улучшенное использование ресурсов
- 3 Балансировка нагрузки на сеть на основании загруженности серверов
- 4 Улучшенная доступность ресурсов
- 5 Уменьшение избыточного потребления ресурсов

[4]

Решения проблемы балансировки нагрузки делятся на 2 основных подхода в зависимости от того, принимает ли алгоритм решения на основе текущего состояния системы или нет: статический и динамический.

Статический подход предполагает наличие знаний о статусе системы, требованиях к ресурсам, времени связи и другим показателям. Балансировка при таком подходе достигается путем создания связей между списком процессоров и списком задач таким образом, чтобы достигалась максимальная производительность. Хотя назначения могут быть как детерминированными, так и вероятностными, текущее состояние системы не учитывается ни в одном из вариантов организации балансировки. Это существенный недостаток статических подходов, который оказывает существенное влияние на общую производительность из-за непредсказуемости колебаний нагрузки на систему. Некоторые методы, используемые при статической балансировке: перечисление и поиск в пространстве решений, теория графов, теория очередей.

При динамическом подходе, решения о балансировке нагрузки принимаются на основе текущего состояния системы; задачам разрешается динамически перемещаться с перегруженного узла на недостаточно загруженный узел для более быстрой обработки. Способность реагировать на изменения в системе является основным преимуществом динамического подхода к балансировке нагрузки. Хотя найти динамическое решение намного сложнее, чем статическое, динамическая балансировка нагрузки может повысить производительность за счет своей адаптивности. Далее сосредоточимся именно на таких подходах.

Механизм управления динамической балансировкой может быть распределенным, когда ответственность за балансировку распределяется между узлами, и нераспределенным, когда от-

ветственность за балансировку лежит на одном или нескольких узлах; включает централизованный (один центральный узел управляет балансировкой всей системы) и полу распределенный (узлы делятся на кластеры с центральными узлами для балансировки внутри кластера) подходы. [1]

2.2 Обзор существующих методов

Рассмотрим методы балансировки нагрузки, применимые к распределённым вычислительным системам с акцентом на гибридные конфигурации (платформы типа BOINC + стабильные кластерные сегменты). Среди критериев для них можно выделить: простота мониторинга, устойчивость к churn (неконтролируемое подключение / отключение узлов), гетерогенность ресурсов, величина накладных расходов, масштабируемость. Всего можно выделить несколько групп подходов к балансировке нагрузки:

- Классические статические методы (Round Robin, Random Strategy) — просты в мониторинге, однако не учитывают размер и время выполнения задач
- Классические динамические методы (Least-Connection) — также просты в мониторинге, но не учитывает общую нагрузку на узлы
- Адаптивные методы — балансировка происходит на основе метрик, вычисляемых в реальном времени; требуют постоянных вычислений
- Предиктивные ML подходы — планировщики, которые хорошо работают даже в сложных условиях, но требуют времени и данных для обучения модели [5]
- Существующие решения для BOINC (CluBoRun) — механизмы верификации, учета доступности, отслеживания churn и другие [3]

Большинство классических и оптимизационных алгоритмов работают при допущении, что узлы доступны и их характеристики относительно стабильны. В средах типа BOINC это допущение неприменимо: изменчивость системы и разнородность устройств приводит к частому перераспределению задач и дополнительным затратам. Если в гомогенных системах перенос задач — относительно стандартный процесс, в гибридных системах такой подход может привести к серьёзному росту накладных расходов из-за неоднородности устройств и отсутствия preemptive migration. Также нужно отметить, что для гибридных систем важно сохранять баланс между количеством собираемых метрик и их полезности для планировщика, что труднодостижимо при подходах с глобальным мониторингом узлов. [4]

2.3 Эвристики планирования задач

Отдельно от сетевых балансировщиков в литературе по планированию задач (job scheduling) рассматриваются эвристики, непосредственно использованные в данной работе:

- **LPT (Longest Processing Time first)** — длинные задачи назначаются первыми; при параллельных однотипных машинах снижает makespan, но может ухудшить fairness и среднее время отклика коротких задач.
- **SJF (Shortest Job First)** — короткие задачи получают приоритет; минимизирует среднее время отклика, но может голодать длинные workunits.
- **Random** — нижняя граница качества планирования; полезен как контрольная политика.

Кластерные планировщики (Slurm, Torque) оптимизированы под гомогенные очереди с централизованным контролем и preemptive scheduling; они не учитывают pull-модель BOINC и добровольный churn узлов, поэтому не применимы «из коробки» к гибридной грид-системе.

2.4 Особенности BOINC

Платформа BOINC использует модель *client-initiated*: вычислительный узел сам запрашивает работу через RPC. Scheduler сканирует shared-memory массив feeder, оценивает совместимость workunit с host и ранжирует кандидатов по score. Назначенная задача не мигрирует на другой host до timeout или повторной репликации. Поле rsc_frops_est задаёт оценку сложности, которая может расходиться с реальным временем выполнения — это важный источник ошибок для любой adaptive-политики. Решение CluBoRun [3] интегрирует кластерные ресурсы в BOINC-проект, но не решает глобальную балансировку по классам устройств (кластер / desktop / mobile).

2.5 Сравнение подходов

Поймем, насколько применимы указанные подходы к гибридным системам и выделим их недочеты:

2.6 Выводы из обзора

- На данный момент существует широкий набор методов балансировки, однако немногие работы систематически исследуют их поведение в гибридных системах

Группа методов	Тип	Требования мониторинга	Цель подхода	Минусы
Классические статические	Простые	Низкие	Простое планирование	Не учитывают размер задач
Классические динамические	Реактивные	Низкие/средние	Сократить число одновременных задач на узле	Не учитывает CPU-time узла
Адаптивные методы	Feedback-driven	Средние/высокие	Низкая задержка и оптимальная нагрузка на узлы	Высокие накладные расходы
Предиктивные подходы	ML-Обучаемые	Высокие	Минимизация целевой функции ошибки	Требуют данных и обучения
CluBORun	Интеграция кластера / устройств	Низкие/средние	Использование idle ресурсов, учёт доступности	Не решает глобальную балансировку

- Основные пробелы: недостаточная оценка влияния churn, упрощённое моделирование сетевых затрат при перераспределении задач, отсутствие оценки накладных расходов мониторинга и планировщика

Данная работа направлена на устранение выявленных ограничений за счёт оптимизации накладных расходов планировщика, более точного учёта динамической доступности узлов, снижения частоты миграций задач и повышения общей производительности системы при рациональном использовании вычислительных ресурсов.

3 Модель гибридной распределённой системы

3.1 Выбор системы моделирования

Для экспериментальной проверки политик балансировки выбран локальный стенд на базе Docker Compose и реального scheduler BOINC 8.3.0. Альтернативы — discrete-event симулятор или GPU-стенд — отвергнуты: симулятор требует отдельной калибровки под протокол BOINC; GPU-модель на Apple Silicon не даёт корректного passthrough в Linux-контейнеры и смещает фокус с CPU-centric volunteer computing.

Основные преимущества выбранного подхода:

- полный RPC-цикл BOINC (feeder, feasibility checks, replication);
- воспроизводимость через скрипты и `manifest.json`;
- контролируемая гетерогенность через Docker cgroup и BOINC preferences;

- `ground truth` по времени выполнения через приложение `uppercase` с параметром `--cpu_time`.

3.2 Топология стенда

Стенд моделирует гибридную систему из шести вычислительных `hosts`:

- `client-cluster`, `client-cluster-2` — сегменты кластера (4 CPU, 4 GiB);
- `client-desktop` — персональный компьютер (2 CPU, 2 GiB);
- `client-low-power` — маломощный узел (1 CPU, 1 GiB);
- `client-phone`, `client-phone-2` — мобильные профили (0.5 CPU, 512 MiB).

Каждый контейнер имеет отдельный BOINC data directory и регистрируется как независимый `host`. Инфраструктура включает MySQL, Apache (scheduler CGI) и сервис сборки проекта.

3.3 Workload и профили churn

Основной набор — 36 задач: 18 short (5 s CPU), 12 medium (30 s), 6 long (120 s). Для коротких smoke-прогонов использовалось 12 задач. Профили доступности:

- **stable** — все узлы online весь прогон;
- **heavy** — у каждого клиента несколько циклов `pause/unpause` с рассинхронизацией (`CHURN_STAGG`).

Churn реализован через паузу контейнера (узел пропадает из сети, но BOINC-процесс не перезапускается). После валидации всех `workunits` churn останавливается.

3.4 Ограничения локального воспроизведения

- масштаб — 6 `hosts` на одной машине, упрощённая сеть;
- BOINC часто сообщает одинаковый `p_flops` для всех `hosts` — в коде добавлен `host_speed_factor` по `hostname`;
- `cgroup CPU` $\neq$ реальное железо (архитектура, энергосбережение);
- `pause/unpause` мягче реального churn (переустановка клиента, смена IP);
- нет preemptive migration задач — как в production BOINC;

- `max_wus_to_send=1, max_wus_in_progress=2` — искусственное ограничение захвата очереди быстрыми клиентами.

Эти ограничения не отменяют ценность А/В-сравнения политик на одном стенде, но не позволяют напрямую экстраполировать абсолютные значения метрик на production BOINC.

4 Реализация системы балансировки

4.1 Точка интеграции

Модификация выполнена в модуле `sched_send.cpp`: при `custom_load_balancer=1` вызывается `send_work_custom()` вместо штатного `send_work_score()`. Feature flag в `config.xml` позволяет проводить А/В-эксперименты без пересборки бинарника.

4.2 Архитектурный принцип

Функция `send_work_custom()` сохраняет весь контракт штатного scheduler: сканирование feeder shared memory, feasibility checks (deadline, память, квоты, replication), locking и обновление БД. **Изменяется только ranking** допустимых кандидатов — минимально инвазивная модификация, снижающая риск нарушить штатное поведение BOINC.

4.3 Экспериментальные политики

Таблица 4.1: Сравнение политик dispatch

Политика	Flag	Суть ranking
baseline	<code>custom_load_balancer=1</code>	Базисный BOINC score: reliability, locality, beta, estimated duration
random	<code>1, random</code>	BOINC score + детерминированный <code>hash(result_id, host_id)</code>
lpt	<code>1, lpt</code>	BOINC score + <code>log(predicted_runtime)</code> — длинные задачи первыми
sjf	<code>1, sjf</code>	BOINC score — <code>log(predicted_runtime)</code> — короткие первыми
hybrid	<code>1, hybrid</code>	Мульти-критериальный score (см. ниже)

Политики `random`, `lpt` и `sjf` — **контрольные**: они изолируют отдельные гипотезы (случайное назначение, минимизация makespan, минимизация response time) и не претендуют на production-решение.

4.4 Hybrid-политика

Для пары (workunit i , host j) вычисляется:

$$\begin{aligned}\hat{t}_{ij} &= \frac{\text{estimate_duration}(i, j)}{\text{host_speed_factor}(j)}, \\ \text{size_affinity}_{ij} &= - \left| \log \frac{\hat{t}_{ij}}{T_{\text{target}}} \right|, \\ \text{deadline_press}_i &= \frac{T_{\text{target}}}{\text{delay_bound}_i}, \\ \text{runtime_penalty}_{ij} &= \log \left(1 + \frac{\hat{t}_{ij}}{T_{\text{target}}} \right), \\ \text{host_fit}_{ij} &= \log(1 + v_j) - \log(1 + \hat{t}_{ij}), \\ \text{score}_{ij} &= 25 \cdot s_{ij}^{\text{BOINC}} + w_{\text{size}} \cdot \text{size_affinity}_{ij} + w_{\text{dl}} \cdot \text{deadline_press}_i \\ &\quad - w_{\text{rt}} \cdot \text{runtime_penalty}_{ij} + 0.5 \cdot w_{\text{size}} \cdot \text{host_fit}_{ij}.\end{aligned}$$

Здесь s_{ij}^{BOINC} — штатный BOINC score, v_j — коэффициент мощности host (cluster=4, desktop=2, low-power=1, phone=0.5), T_{target} — целевое время выполнения (30 s).

Теоретическое обоснование:

- size_affinity и host_fit сопоставляют мощность host и длительность задачи;
- deadline_press предотвращает голодание срочных workunits;
- runtime_penalty снижает назначение длинных задач на медленные hosts;
- множитель $25 \times$ (вместо $100 \times$ у контрольных политик) сохраняет приоритет reliability/locality, но даёт custom-членам влиять на порядок кандидатов.

5 Методика экспериментов

5.1 Протокол

Эксперименты запускались скриптом `experiments/run_experiment.sh all`: подряд выполняются пять политик с одним PAIR_ID. Для каждой политики создаётся свежая БД при одинаковом workload и профиле churn. По завершении экспортируются SQL-данные, рассчитываются метрики и сохраняются в `results/<pair-id>_<policy>/`.

5.2 Группы прогонов

Проведено девять прогонов режима all (по три повтора в каждой группе):

Таблица 5.1: Группы экспериментальных прогонов

Группа	pair_id	Churn	Jobs
A (основная)	20260914T173104Z, 191425Z, 194153Z	stable	36
B (exploratory)	20260914T213013Z, 073506Z, 083656Z	heavy	12
C (stress)	20260915T132706Z, 140620Z, 160921Z	heavy	36

5.3 Сравнимые политики

Внутри каждого прогона сравниваются baseline (штатный BOINC), random, lpt, sjf и hybrid при идентичных условиях. Значения в таблицах ниже — **среднее арифметическое** по трём повторам группы. Во всех прогонах `deadline_miss_rate = 0`.

6 Результаты и анализ

6.1 Stable, 36 задач — основной сценарий

Таблица 6.1: Средние метрики: stable churn, 36 задач

Политика	T	$\bar{R}$, s	R_{95} , s	Makespan, s	Load CV	Jain
baseline	0.121	130.3	183.9	251.7	0.501	0.773
random	0.125	120.2	197.9	248.7	0.463	0.750
lpt	0.126	139.5	201.3	238.3	0.334	0.615
sjf	0.117	116.0	209.7	261.0	0.588	0.778
hybrid	0.117	136.0	214.8	246.0	0.449	0.736

LPT показывает лучшие makespan (≈ 238 s), throughput и баланс нагрузки (Load CV ≈ 0.33): длинные задачи сразу уходят на быстрые cluster-hosts, сокращая хвост набора. Цена — худший Jain (≈ 0.62): cluster-узлы получают непропорционально больше workunits.

Random даёт лучший mean response (≈ 120 s) и высокий throughput: при малом числе hosts отсутствие систематического bias иногда выигрывает у «умного» неправильного предсказания.

SJF минимизирует response time и максимизирует fairness, но ухудшает makespan (≈ 261 s) — long workunits ждут в хвосте.

Baseline — сильный all-rounder без явных провалов; штатный score уже учитывает reliability и estimated duration.

Hybrid на stable-36 **не превосходит baseline** по throughput и makespan; улучшает Load CV относительно baseline, но уступает LPT.

6.2 Heavy churn, 12 задач

Таблица 6.2: Средние метрики: heavy churn, 12 задач

Политика	T	$\bar{R}$, s	Makespan, s	Load CV	Jain
baseline	0.046	91.1	201.0	0.943	0.816
random	0.049	96.7	189.0	0.810	0.699
lpt	0.047	107.1	204.3	0.758	0.816
sjf	0.049	84.9	190.0	0.908	0.752
hybrid	0.045	104.0	205.7	0.676	0.838

При малом числе задач наблюдается высокая дисперсия; абсолютные значения несопоставимы с 36-job группами. **Hybrid** лидирует по Jain и Load CV, но проигрывает по throughput. Группа В годится как иллюстрация влияния churn, но не как основное доказательство.

6.3 Heavy churn, 36 задач

Таблица 6.3: Средние метрики: heavy churn, 36 задач

Политика	T	$\bar{R}$, s	R_{95} , s	Makespan, s	Load CV	Jain
baseline	0.097	157.1	233.5	324.3	0.558	0.787
random	0.095	152.5	235.1	324.0	0.471	0.782
lpt	0.095	166.2	215.8	261.3	0.263	0.620
sjf	0.101	134.7	235.0	302.0	0.379	0.817
hybrid	0.088	154.5	227.3	349.7	0.599	0.923

Сценарий наиболее стрессовый и нестабильный (высокий разброс между прогонами). **LPT** минимизирует makespan и Load CV, когда «работает»; в отдельных прогонах throughput падал из-за совмещения long jobs с offline cluster-hosts.

Hybrid стабильно даёт лучший Jain (≈ 0.923) — почти равномерное распределение завершённых задач по шести hosts; это следствие host_fit и size_affinity. Однако hybrid проигрывает по makespan (≈ 350 s vs ≈ 324 s у baseline) и throughput: при heavy churn медленные hosts получают workunits, пока быстрые offline.

Таблица 6.4: Качественное сравнение политик по сценариям

Политика	stable-36	heavy-36
LPT	скорость ✓, fairness ×	скорость ~, fairness ×
Hybrid	скорость ~, fairness ~	скорость ×, fairness ✓✓
Baseline	скорость ✓, fairness ✓	скорость ✓, fairness ✓

6.4 Сводный trade-off

6.5 Выводы по экспериментам

- 1 Разработан и интегрирован кастомный dispatch layer для BOINC с пятью режимами сравнения.
- 2 На stable-36 **LPT** лучше по makespan, throughput и балансу нагрузки; **baseline** — надёжная база без провалов.
- 3 **Hybrid** улучшает fairness при churn, но ухудшает throughput/makespan из-за назначения работы ослабленным hosts в моменты их кратковременной доступности.
- 4 Предложенная политика — проверяемая гипотеза, а не гарантированное улучшение; требуется настройка весов или учёт availability.

7 Заключение

В рамках курсовой работы выполнены все поставленные задачи:

- 1 проведён обзор методов балансировки для кластеров и распределённых систем с акцентом на BOINC и гибридные конфигурации;
- 2 выбрана и адаптирована CPU-модель гибридной системы на Docker/BOINC с шестью профилями hosts и управляемым churn;
- 3 реализована модификация sched_send с политиками baseline, random, lpt, sjf и hybrid;
- 4 проведено девять сравнительных прогонов; основной вывод — LPT минимизирует makespan на stable-стенде, hybrid улучшает fairness при churn ценой скорости.

Ограничения: малый масштаб (6 hosts), упрощённый churn, hostname-based speed hints, отсутствие GPU plan class.

Направления дальнейшей работы: учёт битовой availability в score, segment-aware routing, sensitivity analysis весов hybrid-политики, per-class breakdown (short/medium/long) по типам hosts.

Список литературы

- [1] Ali M. Alakeel. «A Guide to Dynamic Load Balancing in Distributed Computer Systems». B: *International Journal of Computer Science and Network Security* 1.2 (2010).
- [2] Tony Bourke. *Server Load Balancing*. O'Reilly & Associates, 2001.
- [3] Dolgov A. Kurochkin I. «Load balancing in grid systems from personal devices on the BOINC platform». B: *XIV All-russian multi-conference on management issues of the MCPU-2021*. 2021.
- [4] Jerry Gao Mazedur Rahman Samira Iqbal. «Load Balancer as a Service in Cloud Computing». B: *IEEE 8th International Symposium on Service Oriented System Engineering (SOSE)*. 2014.
- [5] Lun WANG. «Low-Latency, High-Throughput Load Balancing Algorithms». B: *Journal of Computer Technology and Applied Mathematics* 10.6 (2024), с. 153—160.